

Interview Task: Financial One-Pager Extractor and Research Chatbot

Context

Build a working prototype of a Geojit-inspired financial research workflow. The user uploads a company-related financial document, the app extracts the important information, enriches it with InsightSentry data where relevant, and presents a compact financial one-pager with a sidebar chatbot.

The experience should feel like a credible finance product: structured, data-led, easy to scan, and useful for an investor, analyst, or relationship manager. Do not copy Geojit assets or proprietary layouts directly; use that style only as a reference for density, clarity, and financial presentation.

Candidates may use Codex or any other AI coding assistant while building the solution.

InsightSentry Context

InsightSentry is a financial data API with REST and WebSocket access, real-time and historical market data, fundamentals, news feed, screeners, options data, and document tools for filings and transcripts where available.

For this task, use InsightSentry to enrich and validate company context through filings, transcripts, fundamentals, quotes, market data, and news where coverage is available. Treat uploaded document parsing as part of your application.

Objective

Create a working application where a user can:

1. Upload a company document such as an annual report, quarterly result, investor presentation, exchange filing, brokerage note, or press release.
2. Extract key information from the uploaded document.
3. Detect the company or ticker where possible.
4. Fetch relevant company context from InsightSentry, or from a clearly documented mock.
5. Generate a polished one-page financial summary.
6. Ask questions in a sidebar chatbot grounded in the generated one-pager.

7. Provide a new article, filing update, press release, or management comment, then have the LLM decide which one-pager sections should update, change only those sections, and show a brief rationale, evidence, and audit trail.

Expected Product Flow

The app should start with a simple upload state. After upload, show clear progress through extraction, enrichment, summary generation, and completion. A basic error and retry state is enough.

The generated one-pager should be the main surface. The chatbot should sit in a sidebar and feel connected to the one-pager, not like a separate generic chat tool.

For the update workflow, the user should be able to paste text or upload a small related update. The system should compare that update against the existing one-pager and propose surgical changes.

One-Pager Content

Design the one-pager like a financial research product, not a chat transcript. Use the sections that make sense for the uploaded document and available API context. Suggested sections:

- Company snapshot
- Business overview
- Key financial highlights
- Growth, margin, or profitability commentary
- Segment or product insights, if available
- Recent developments
- Key risks
- Analyst-style takeaway
- Sources used

The summary should be concise, scannable, and credible. Use numbers only when supported by the uploaded document or API response. If information is missing or uncertain, say so plainly.

Sidebar Chatbot

The chatbot should answer using the generated one-pager as its primary context.

Example questions:

- "What are the top risks?"
- "What changed recently?"
- "Is the outlook positive or cautious?"
- "Which claims are source-backed?"
- "What should an investor watch next?"

Expected behavior:

- Ground answers in the one-pager.
- Refer to the relevant one-pager section when useful.
- Say when the one-pager does not contain enough information.
- Avoid silently using unsupported external knowledge.

Surgical Update Feature

This is the most important update feature in the task.

When the user provides a new article, filing excerpt, press release, or management update, compare it with the current one-pager and produce:

- Impacted sections
- Before and after text for each changed section
- Brief rationale and evidence for each change
- Sections that were reviewed but left unchanged
- Timestamped audit trail

Do not regenerate the full one-pager blindly. The goal is controlled, explainable updates.

Example: if the one-pager says "Debt remains elevated" and a new filing says the company completed a major repayment, the system might update financial highlights, risks, and analyst takeaway. It should also explain why the business overview may remain unchanged.

Technical Expectations

Use any stack you prefer. Keep the API and LLM boundaries clean. A reasonable structure might include:

- `extractUploadedDocument(file)`
- `fetchInsightSentryContext(companyOrTicker)`
- `generateOnePager(extractedContent, apiContext)`
- `answerFromOnePager(onePager, question)`
- `proposeSectionUpdates(onePager, newInput)`

If InsightSentry credentials are unavailable, mock the InsightSentry adapter with realistic sample responses. The mock should be easy to replace with the real API later. The uploaded document flow should still work with at least basic text extraction or a clearly documented sample-document path.

The prototype does not need authentication, payments, or production deployment.

Submission

Submit a repo or zip with:

- Setup instructions
- Sample document or sample input flow
- README covering architecture, API/mocking choices, LLM prompting strategy, known limitations, and what you would improve with more time
- Any required environment variables, without secret values

Evaluation

We will review:

- Product clarity and finance-grade UX
- Quality of the generated one-pager
- Grounding and refusal behavior in the chatbot
- Surgical-update rationale and evidence
- Usefulness of the audit trail
- Code organization and API boundaries
- Ease of running the app

A focused, thoughtful prototype is better than a broad but fragile implementation.